GROUP C · OBJECT-ORIENTED PROGRAMMING MINI PROJECT

STOCKROOM

Inventory Management System using Java Object-Oriented Programming

A browser-based inventory application implementing product listing, product details, purchase, shipping, stock balance, profit and loss calculation, and date-range transaction reporting.

Core language: Java 17

Architecture: Java HTTP API + responsive HTML/CSS/JavaScript frontend

Deployment: Docker container on Render

Repository: github.com/Akram2903/INVETARY_SYSTEM

1. Project Overview

Stockroom is an inventory management system created from the Group C problem statement.

The system maintains a catalogue of products with product IDs, names, categories, stock quantities, cost prices, and selling prices. It provides operations for incoming stock through purchases and outgoing stock through shipping or sales. The Java backend owns all inventory rules and calculations, while the frontend presents the catalogue, dashboard, action form, and reports in a browser.

Java 17	6	REST	Docker
OOP core	Seed products	HTTP API	Render deployment

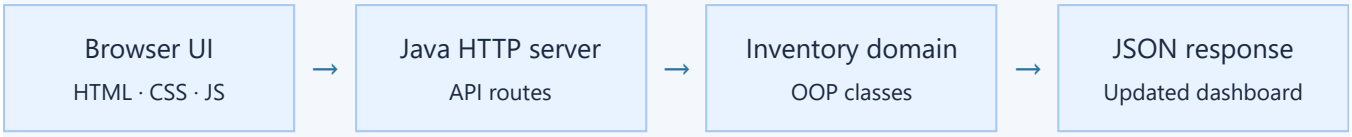
Required operations implemented

- List all products.
- Display individual product information in the catalogue.
- Purchase or restock products.
- Ship or sell products.
- Balance stock after every movement.
- Calculate inventory value, revenue, margin, and profit.
- Generate purchase and shipping reports between dates.

Project objectives

- Apply Java classes, objects, encapsulation, collections, constructors, and methods to a practical system.
- Separate domain logic from HTTP and presentation concerns.
- Prevent invalid stock operations such as zero quantities and shipping more than available stock.
- Make the result interactive and deployable for browser demonstration.

2. System Architecture



Request flow

1. The browser requests the product catalogue and dashboard when the page loads.
2. The Java server returns JSON from the in-memory `Inventory` object.
3. When a purchase or shipping form is submitted, JavaScript sends a POST request.
4. The server validates the action, finds the product, checks the quantity and stock, and changes the product stock.
5. A `Transaction` object is created and stored.
6. The server returns a JSON result; the frontend reloads products and dashboard totals.

Layer	Files	Responsibility
Domain	`InventoryServer.java`	`Product`, `Transaction`, and `Inventory` classes contain the OOP model and business operations.
HTTP	`InventoryServer`	Routes requests, parses query/form data, serves JSON, and serves frontend files.
Frontend	`public/index.html`, `app.js`, `styles.css`	Dashboard, catalogue, search, movement form, report filter, and responsive presentation.
Deployment	`Dockerfile`, `render.yaml`	Compiles Java in a JDK image and runs it in a Java runtime image on Render.

3. Java OOP Design

Product class

```
static final class Product {
    private final String id;
    private final String name;
    private final String category;
    private int stock;
    private final double costPrice;
    private final double sellingPrice;
}
```

A `Product` represents one inventory item. Its identity and price fields are final after construction. Stock is mutable because purchases and shipments change it. The private fields demonstrate encapsulation: external code does not directly manipulate the product data.

Field	Meaning
`id`	Unique product identifier such as P1001.
`name`	Human-readable product name.
`category`	Product grouping such as Audio or Storage.
`stock`	Current number of available units.
`costPrice`	Amount paid to acquire one unit.
`sellingPrice`	Amount charged for one shipped/sold unit.

Transaction class

A `Transaction` records an inventory movement. It stores the operation type, the product object, quantity, customer or supplier label, total amount, and the date. Purchase transactions use cost price, while shipping transactions use selling price.

Inventory class

`Inventory` owns two collections: a product catalogue and a transaction history. It is the main business service. Its methods add products, process movements, serialize products, calculate dashboard totals, and create date-filtered reports.

InventoryServer class

`InventoryServer` is the application entry point. It creates the Java HTTP server, seeds initial data, registers URL contexts, and connects HTTP requests to the inventory domain.

4. Product Initialization and Java Collections

```
private static void seedProducts() {
    inventory.addProduct(new Product(
        "P1001", "Wireless Headphones", "Audio",
        85, 49.99, 79.99
    ));
    inventory.addProduct(new Product(
        "P1002", "Mechanical Keyboard", "Accessories",
        42, 58.00, 94.99
    ));
}
```

The server starts with six sample products so the dashboard is immediately useful after deployment. Each constructor call creates a Product object and passes it to the Inventory catalogue.

```
private final List<Product> products = new ArrayList<>();
private final List<Transaction> transactions = new ArrayList<>();
```

`ArrayList` provides dynamic storage for the catalogue and history. Stream operations search, filter, map, sort, and reduce these collections. This keeps the implementation concise while still expressing the inventory rules clearly.

Product lookup

```
Product product = products.stream()
    .filter(item -> item.id.equals(productId))
    .findFirst()
    .orElseThrow(() -> new IllegalArgumentException(
        "Product not found"));
```

The stream checks every product until the requested ID is found. If no product matches, an explicit exception is raised and converted into a clear HTTP 400 error.

5. Purchase and Shipping Logic

```
synchronized Transaction process(
    String action, String productId,
    int quantity, String customer) {
    if (quantity <= 0)
        throw new IllegalArgumentException(
            "Quantity must be greater than zero");

    Product product = find product by ID;
    if ("shipping".equals(action)
        && product.stock < quantity)
        throw new IllegalArgumentException(
            "Insufficient stock");

    if ("purchase".equals(action))
        product.stock += quantity;
    else
        product.stock -= quantity;
}
```

Purchase/restock

A purchase represents stock arriving from a supplier. The quantity is added to the product stock. The total purchase amount is calculated using `quantity × costPrice`. A transaction is then saved.

Shipping/sale

A shipment represents stock leaving the warehouse for a customer. The method first verifies that enough stock exists. Only then is the quantity subtracted. The total sale amount is calculated using `quantity × sellingPrice`.

Why the method is synchronized

The method is synchronized so two simultaneous requests cannot update the same inventory state at the same time in an unsafe order. This is a simple concurrency protection for the in-memory domain model.

Transaction creation

```
double total = quantity * (
    "purchase".equals(action)
        ? product.costPrice
        : product.sellingPrice
);
Transaction transaction = new Transaction(
    action, product, quantity, customer, total
);
transactions.add(transaction);
return transaction;
```

6. Stock Balance and Profit/Loss

Stock balance

The current stock is stored directly on each Product object. Purchases increase it and shipments decrease it. The dashboard calculates total units with:

```
int totalUnits = products.stream()
    .mapToInt(item -> item.stock)
    .sum();
```

Inventory value

Inventory value estimates the cost invested in stock still held:

```
double inventoryValue = products.stream()
    .mapToDouble(item -> item.stock * item.costPrice)
    .sum();
```

Revenue

Revenue includes only shipping transactions because a shipment is treated as a completed sale:

```
double revenue = transactions.stream()
    .filter(item -> item.type.equals("shipping"))
    .mapToDouble(item -> item.total)
    .sum();
```

Profit

The cost of shipped goods is calculated using the product cost price. Profit is revenue minus that cost:

```
double cost = transactions.stream()
    .filter(item -> item.type.equals("shipping"))
    .mapToDouble(item ->
        item.quantity * item.product.costPrice)
    .sum();
double profit = revenue - cost;
```

This is a basic gross profit calculation. It does not include salaries, shipping expenses, taxes, discounts, or other operating costs.

Low-stock detection

```
long lowStock = products.stream()
    .filter(item -> item.stock < 25)
    .count();
```

Products with fewer than 25 units are displayed as low-stock items. The frontend uses a different color for these products.

7. Purchase and Transaction Reports

```
synchronized String reportJson(String from, String to) {
    LocalDate start = from == null || from.isBlank()
        ? LocalDate.MIN : LocalDate.parse(from);
    LocalDate end = to == null || to.isBlank()
        ? LocalDate.MAX : LocalDate.parse(to);

    return transactions.stream()
        .filter(item -> !item.date.isBefore(start)
            && !item.date.isAfter(end))
        .sorted(Comparator.comparing(item -> item.date))
        .map(Transaction::json)
        .reduce((a, b) -> a + "," + b)
        .orElse("");
}
```

The report accepts optional ISO dates in `YYYY-MM-DD` format. Missing dates mean no lower or upper restriction. Transactions are included when their date is between the selected start and end dates, inclusive. The output is sorted chronologically.

Report fields

- Transaction type: purchase or shipping.
- Product name.
- Quantity moved.
- Total amount.
- Transaction date.

Frontend date filter

The browser sends `view=report`, `from`, and `to` query values. It renders each returned transaction in the report panel. If no records match, it shows an empty-state message.

8. Java HTTP Server and API

Server creation

```
int port = Integer.parseInt(
    System.getenv().getOrDefault("PORT", "3000"));
HttpServer server = HttpServer.create(
    new InetSocketAddress(port), 0);
server.createContext("/api/inventory",
    InventoryServer::handleInventory);
server.createContext("/api/health",
    InventoryServer::handleHealth);
server.createContext("/",
    InventoryServer::serveFrontend);
server.setExecutor(
    Executors.newFixedThreadPool(8));
server.start();
```

The project uses Java's built-in `com.sun.net.httpserver.HttpServer`, so no external backend framework is required. Render supplies a PORT environment variable, while local execution defaults to 3000. A fixed thread pool allows the server to handle multiple requests.

API endpoints

Method	Endpoint	Purpose
GET	`/api/inventory`	Dashboard totals: product count, units, value, revenue, profit, low stock, transactions.
GET	`/api/inventory?view=products`	Returns all product information.
GET	`/api/inventory?view=report&from=...&to=...`	Returns date-filtered transaction history.
POST	`/api/inventory`	Processes purchase or shipping form data.
GET	`/api/health`	Render health check.

POST data

```
action=purchase|shipping
productId=P1001
quantity=5
customer=Supplier or customer name
```

9. Request Parsing and Error Handling

```
private static Map<String, String> parseQuery(String query) {
    Map<String, String> values = new HashMap<>();
    if (query == null || query.isBlank()) return values;
    for (String pair : query.split("&")) {
        String[] parts = pair.split("=", 2);
        String key = decode(parts[0]);
        String value = parts.length > 1
            ? decode(parts[1]) : "";
        values.put(key, value);
    }
    return values;
}
```

The server parses both URL query parameters and URL-encoded POST bodies. URL decoding allows spaces and special characters to be represented correctly.

Status behavior

Condition	Response
Valid dashboard, products, report, or movement	HTTP 200 with JSON data.
Invalid action, product, or quantity	HTTP 400 with an error message.
Unsupported HTTP method	HTTP 405.
Unexpected server failure	HTTP 500 with a generic error and server-side stack trace.
Unknown frontend file	HTTP 404.

Frontend file safety

```
Path file = publicDirectory
    .resolve(requested.substring(1)).normalize();
if (!file.startsWith(publicDirectory)
    || !Files.exists(file)
    || Files.isDirectory(file)) {
    // 404
}
```

The normalized path check prevents a request from escaping the public directory through path traversal segments.

10. Frontend Implementation

Dashboard and catalogue

The frontend calls the dashboard and products endpoints on startup. It displays product count, units in stock, inventory value, profit, low-stock count, category, product ID, selling price, and per-unit margin.

```
async function loadDashboard() {
    const response = await fetch('/api/inventory');
    const data = await response.json();
    document.querySelector('#products').textContent = data.products;
    document.querySelector('#units').textContent = data.units;
    document.querySelector('#value').textContent =
        money(data.inventoryValue);
    document.querySelector('#profit').textContent =
        money(data.profit);
}
```

Search

Search works in the browser by matching the entered text against product name, category, and ID. It filters the already-loaded catalogue without another server request.

Movement form

The form sends URL-encoded data to the Java API. A successful movement reloads product stock and all dashboard totals. An error is shown in the notice area without changing the UI to a false success state.

HTML escaping

Product names and report values are escaped before being inserted into HTML. This prevents user-controlled text from being interpreted as markup.

11. User Interface

The interface is designed as an operations dashboard with a calm blue and mint palette. It separates information, catalogue browsing, warehouse actions, and reporting.

- Top metrics provide an immediate inventory summary.
- The catalogue shows products in searchable cards.
- The action panel handles purchase and shipping.
- The report panel accepts a from/to date range.
- Low-stock values use a warning color.
- The layout becomes a single column on smaller screens.

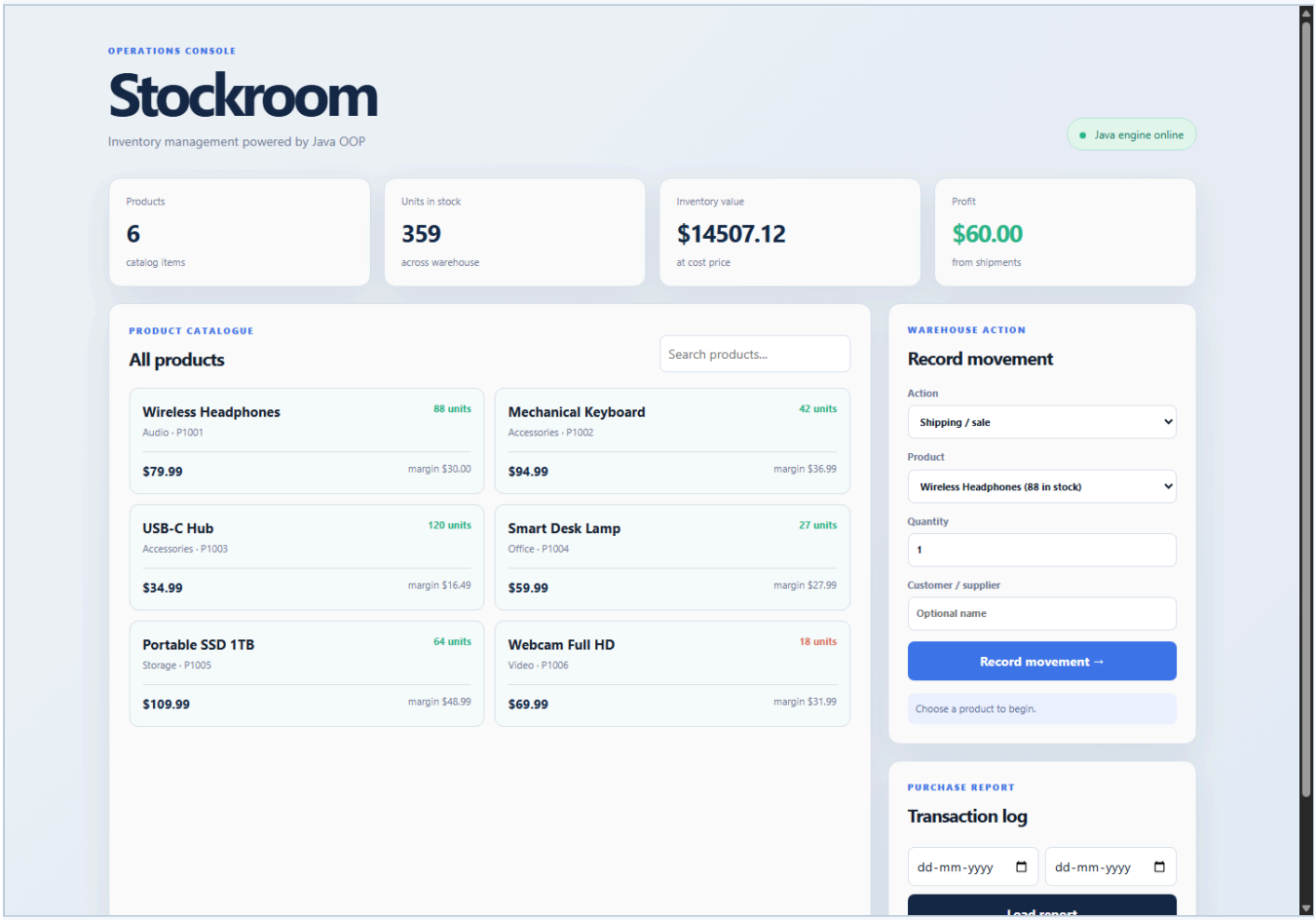


Figure 1. Stockroom dashboard showing product catalogue, inventory metrics, warehouse actions, and report controls.

12. Technology Stack and Project Structure

Technology	Role
Java 17	OOP domain model, inventory rules, calculations, HTTP server.
Java Collections	ArrayList and streams for product and transaction management.
HTML5	Dashboard structure and forms.
CSS3	Responsive dashboard styling and visual states.
JavaScript	Fetch requests, search, dynamic cards, form submission, report rendering.
Docker	Reproducible Java build and runtime environment.
Render	Public web deployment and health checks.
GitHub	Source repository: Akram2903/INVETARY_SYSTEM.

```
inventory-management-java/
├── Dockerfile
├── render.yaml
├── pom.xml
├── README.md
├── public/
│   ├── index.html
│   ├── styles.css
│   └── app.js
└── src/main/java/com/inventory/
    └── InventoryServer.java
```

13. Docker and Render Deployment

Multi-stage Dockerfile

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY src ./src
RUN mkdir -p target/classes && \
    javac -d target/classes \
    src/main/java/com/inventory/InventoryServer.java

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/classes ./target/classes
COPY public ./public
CMD ["java", "-cp", "target/classes",
     "com.inventory.InventoryServer"]
```

The first image contains a JDK and compiles the source. The second image contains only the Java runtime and compiled classes plus frontend files. This separates build tools from the production runtime.

Render configuration

```
services:
  - type: web
    name: java-inventory-management
    runtime: docker
    dockerfilePath: ./Dockerfile
    healthCheckPath: /api/health
```

Render builds the Dockerfile, starts the Java process, and calls the health endpoint. No manual environment variable is needed. Render's PORT is read by the Java server automatically.

Deployment correction

The initial Render attempt used `mvn clean package` while Render had been configured as Node, producing `mvn: command not found`. The project was corrected to Docker, which provides the required JDK and JRE images consistently.

14. Complexity and Design Analysis

Operation	Complexity	Reason
Find product	O(P)	Products are stored in an ArrayList and searched linearly by ID.
Purchase/shipping	O(P)	Lookup is linear; stock update and transaction append are O(1).
Dashboard	O(P + T)	Streams scan products and transactions.
Report	O(T log T)	Transactions are filtered and sorted by date.
Space	O(P + T)	Catalogue and transaction history remain in memory.

Here, P is the number of products and T is the number of transactions. For a larger production system, a database and indexed product lookup could improve persistence and scalability. The in-memory design is appropriate for a focused OOP mini project.

Encapsulation and responsibility

The Inventory class owns business state and rules. The Product class owns product data. The Transaction class owns movement records. The server translates HTTP requests but does not implement stock mathematics itself. This separation makes the design easier to understand and extend.

15. Testing and Verification

Test	Expected result	Result
Compile Java source	Classes generated successfully.	Passed with javac.
List products	Six seeded products returned.	Passed.
Dashboard	Initial total of 356 units returned.	Passed.
Purchase P1001 × 5	Stock increases and purchase transaction is saved.	Passed.
Shipping P1001 × 2	Stock decreases and shipment is saved.	Passed.
Report	Transactions returned by date range.	Passed.
Excessive shipping	Request rejected with insufficient-stock error.	Passed.
Health endpoint	HTTP 200 and status OK.	Passed.
Homepage	HTTP 200 and dashboard HTML served.	Passed.

Example sequence

```
Initial stock: P1001 = 85
Purchase 5:    P1001 = 90
Shipping 2:    P1001 = 88
Profit:        revenue from shipping - cost of shipped units
Invalid 999:   rejected because available stock is insufficient
```

16. Technical Questions and Answers

Q1. What is the purpose of this project?
It manages products, stock movements, inventory value, profit, and transaction reports through a browser interface backed by Java OOP logic.

Q2. Which OOP concepts are used?
Classes and objects, encapsulation through private fields, constructors, composition between Inventory and Product/Transaction objects, methods, and separation of responsibilities.

Q3. What is the Product class?
It is the domain representation of one catalogue item, containing identity, category, stock, cost price, and selling price.

Q4. What is the Transaction class?
It records a purchase or shipping event with type, product, quantity, customer/supplier, total amount, and date.

Q5. What does Inventory do?
It owns the product and transaction collections and performs lookup, stock changes, dashboard calculations, and reporting.

Q6. What happens during a purchase?
The requested product is found, quantity is validated, stock is increased, cost-based total is calculated, and a purchase transaction is stored.

Q7. What happens during shipping?
The product is found, quantity is validated against available stock, stock is decreased, selling-price total is calculated, and a shipping transaction is stored.

Q8. How is insufficient stock handled?
The operation throws an IllegalArgumentException before changing stock. The HTTP layer returns HTTP 400 with a descriptive message.

Q9. How is profit calculated?
Profit equals revenue from shipments minus the cost of the units shipped. Revenue uses selling price and cost uses cost price.

Q10. Is inventory value the same as revenue?
No. Inventory value is the cost value of stock still held. Revenue is money from completed shipments.

Q11. Why is stock mutable but prices final?
Stock changes during operations, while the seeded product prices remain fixed for this project. A production system could support controlled price updates.

Q12. Why is the method synchronized?
To prevent concurrent requests from interleaving stock updates unsafely in the shared in-memory Inventory object.

Q13. Why use ArrayList?
It provides dynamic storage and convenient iteration and stream operations. It is adequate for the scale of this mini project.

Q14. What is the time complexity of a product lookup?
O(P), because the ArrayList is searched linearly by product ID.

Q15. What is the time complexity of reporting?
Filtering is O(T) and sorting matching transactions is O(T log T) in the worst case.

Q16. Why use Java's built-in HttpServer?
It keeps the deployment lightweight and demonstrates that Java can provide an HTTP API without an external framework.

Q17. What does the frontend do?
It displays JSON data, submits purchase/shipping forms, filters products, reloads dashboard data, and displays reports.

Q18. Why is HTML escaping used?
Product and transaction text is displayed in generated HTML. Escaping prevents special input from being interpreted as markup.

Q19. Why is Docker used on Render?
The Render dashboard did not provide a native Java runtime option for this service. Docker supplies a Java 17 JDK for compilation and JRE for execution.

Q20. What is the purpose of /api/health?
Render calls it to confirm that the server has started and is responding successfully.

Q21. Is data persistent?
No. The current version stores data in memory, so restarting the service resets the seeded catalogue and transactions. A database can be added as an extension.

Q22. How could the project be improved?

Add a database, authentication, product CRUD, supplier/customer entities, pagination, indexed lookup, audit logs, and separate service/repository packages.

17. Conclusion

Stockroom applies object-oriented Java programming to a complete inventory workflow. Product and transaction objects are managed by an Inventory service that enforces stock rules and calculates operational values. A lightweight Java HTTP server exposes that logic to an interactive browser dashboard.

The project covers the requested inventory operations while demonstrating encapsulation, collections, stream processing, validation, concurrency awareness, API design, responsive frontend development, and Docker-based cloud deployment.

> **Central result:** the system maintains an accurate in-memory inventory state, records purchases and shipments, prevents invalid stock movement, calculates financial summaries, and exposes all functionality through a deployable web interface.

Repository: github.com/Akram2903/INVETARY_SYSTEM